

fixed_capacity_vector

A dynamically-resizable vector with fixed capacity and embedded storage (revision 0)

Document number: P0843.

Date: 2017-10-15.

Project: Programming Language C++, Library Working Group.

Audience: LWG/LEWG.

Reply-to: Gonzalo Brito Gadeschi <gonzalo.gadeschi at rwth-aachen dot de>.

Table of contents

- [1. Introduction](#)
- [2. Motivation](#)
- [3. Existing practice](#)
- [4. Design Decisions](#)
 - [4.1 Storage/Memory Layout](#)
 - [4.2 Move semantics](#)
 - [4.3 `constexpr` support](#)
 - [4.4 Exception safety](#)
 - [4.5 Iterator invalidation](#)
 - [4.6 Naming](#)
 - [4.7 Potential extensions](#)
- [5. Technical Specification](#)
 - [5.1 Overview](#)
 - [5.2 Construction](#)
 - [5.3 Destruction](#)
 - [5.4 Size and capacity](#)
 - [5.5 Element and data access](#)
 - [5.6 Modifiers](#)
 - [5.7 Specialized algorithms](#)
 - [5.8 Zero sized `fixed\_capacity\_vector`](#)
 - [5.9 Size type](#)
- [6. Acknowledgments](#)
- [7. References](#)

1. Introduction

This paper proposes a modernized version of `boost::container::static_vector<T, Capacity>` [1]. That is, a dynamically-resizable `vector` with compile-time fixed capacity and contiguous embedded storage in which the vector elements are stored within the vector object itself.

Its API closely resembles that of `std::vector<T, A>`. It is a contiguous container with `O(1)` insertion and removal of elements at the end (non-amortized) and worst case `O(size())` insertion and removal otherwise. Like `std::vector`, the elements are initialized on insertion and destroyed on removal. For trivial `value_type`s, the vector is fully usable inside `constexpr` functions.

2. Motivation and Scope

The `fixed_capacity_vector` container is useful when:

- memory allocation is not possible, e.g., embedded environments without a free store, where only a stack and the static memory segment are available,
- memory allocation imposes an unacceptable performance penalty, e.g., with respect to latency,
- allocation of objects with complex lifetimes in the *static*-memory segment is required,
- `std::array` is not an option, e.g., if non-default constructible objects must be stored,
- a dynamically-resizable array is required within `constexpr` functions,
- the storage location of the vector elements is required to be within the vector object itself (e.g. to support `memcpy` for serialization purposes).

3. Existing practice

There are at least 3 widely used implementations of `fixed_capacity_vector`: [Boost.Container](#) [1], [EASTL](#) [2], and [Folly](#) [3]. The main difference between these is that `Boost.Container` implements `fixed_capacity_vector` as a standalone type with its own guarantees, while both EASTL and Folly implement it by adding an extra template parameter to their `small_vector` types.

A `fixed_capacity_vector` can also be poorly emulated by using a custom allocator, like for example [Howard Hinnant's `stack\_alloc`](#) [4], on top of `std::vector`.

This proposal shares a similar purpose with [P0494R0](#) [5] and [P0597R0](#): `std::constexpr_vector<T>` [6]. The main difference is that this proposal closely follows `boost::container::static_vector` [1] and proposes to standardize existing practice. A prototype implementation of this proposal for standardization purposes is provided here: http://github.com/gnzlbq/fixed_capacity_vector.

4. Design Decisions

The most fundamental question that must be answered is:

Should `fixed_capacity_vector` be a standalone type or a special case of some other type?

The [EASTL](#) [2] and [Folly](#) [3] special case `small_vector`, e.g., using a 4th template parameter, to make it become a `fixed_capacity_vector`. The paper [P0639R0: Changing attack vector of the `constexpr\_vector`](#) [7] proposes improving the `Allocator` concepts to allow `fixed_capacity_vector`, among others, to be implemented as a special case of `std::vector` with a custom allocator.

Both approaches run into the same fundamental issue: `fixed_capacity_vector` methods are identically-named to those of `std::vector` yet they have subtly different effects, exception-safety, iterator invalidation, and complexity guarantees.

This proposal follows `boost::container::static_vector<T,Capacity>` [1] closely and specifies the semantics that `fixed_capacity_vector` ought to have as a standalone type. As a library component this delivers immediate value.

I hope that having the concise semantics of this type specified will also be helpful for those that want to generalize the `Allocator` interface to allow implementing `fixed_capacity_vector` as a `std::vector` with a custom allocator.

4.1 Storage/Memory Layout

The container models `ContiguousContainer`. The elements of the vector are contiguously stored and properly aligned within the vector object itself. The exact location of the contiguous elements within the vector is not specified. If the `Capacity` is zero the container has zero size:

```
static_assert(sizeof(fixed_capacity_vector<int, 3>) == 0);
```

This optimization is easily implementable, enables the EBO, and felt right.

4.2 Move semantics

The move semantics of `fixed_capacity_vector<T, Capacity>` are equal to those of `std::array<T, Size>`. That is, after

```
fixed_capacity_vector a(10);
fixed_capacity_vector b(std::move(a));
```

the elements of `a` have been moved element-wise into `b`, the elements of `a` are left in an initialized but unspecified state (have been moved from state), the size of `a` is not altered, and `a.size() == b.size()`.

Note: this behavior differs from `std::vector<T, Allocator>`, in particular for the similar case in which `std::propagate_on_container_move_assignment<Allocator>{} is false`. In this situation the state of `std::vector` is initialized but unspecified.

4.3 constexpr support

The API of `fixed_capacity_vector<T, Capacity>` is `constexpr`. If `is_trivial_v<T>` is `true`, `fixed_capacity_vector`s can be seamlessly used from `constexpr` code. This allows using `fixed_capacity_vector` as a `constexpr_vector` to, e.g., implement other `constexpr` containers.

The implementation cost of this is very low for trivial types: the implementation can use an array for `fixed_capacity_vector`'s storage without altering the guarantees of `fixed_capacity_vector`'s methods in an observable way.

For example, `fixed_capacity_vector(N)` constructor guarantees a complexity "Linear in `N`". Strictly speaking, `fixed_capacity_vector`'s constructor for trivial types will construct an array of `Capacity` length. However, because `is_trivial_v<T>` is `true`, the number of constructor or destructor calls is not observable.

4.4 Exception Safety

The only operations that can actually fail within `fixed_capacity_vector<T, Capacity>` are:

1. `value_type`'s constructors/assignment/destructors/swap can potentially throw,
2. Mutating operations exceeding the capacity (`push_back`, `insert`, `emplace`, `pop_back` when `empty()`, `fixed_capacity_vector(T, size)`, `fixed_capacity_vector(begin, end)` ...).
3. Out-of-bounds unchecked access: 2.1 `front` / `back` / `pop_back` when empty, `operator[]` (unchecked random-access). 2.2 `at` (checked random-access) which can throw `out_of_range` exception.

When `value_type`'s operations are invoked, the exception safety guarantees of `fixed_capacity_vector` depend on whether these operations can throw. This is detected with `noexcept`.

Since its `Capacity` is fixed at compile-time, `fixed_capacity_vector` never dynamically allocates memory, the answer to the following question determines the exception safety for all other operations:

What should `fixed_capacity_vector` do when its `Capacity` is exceeded?

Two main answers were explored in the prototype implementation:

1. Throw an exception.
2. Make this a precondition violation.

Throwing an exception is appealing because it makes the interface slightly more similar to that of `std::vector`. However, which exception should be thrown? It cannot be `std::bad_alloc`, because nothing is being allocated. It could throw either `std::out_of_bounds` or `std::logic_error` but in any case the interface does not end up being equal to that of `std::vector`.

The alternative is to make not exceeding the capacity a precondition on the vector's methods. This approach allows implementations to provide good run-time diagnostics if they so desired, e.g., on debug builds by means of an assertion, and makes implementation that avoid run-time checks conforming as well. Since the mutating methods have a precondition, they have narrow contracts, and are not conditionally `noexcept` .

This proposal chooses this path and makes exceeding the vector's capacity a precondition violation that results in undefined behavior. Throwing `checked_xxx` methods can be provided in a backwards compatible way.

4.5 Iterator invalidation

The iterator invalidation rules are different than those for `std::vector` , since:

- moving a `fixed_capacity_vector` invalidates all iterators,
- swapping two `fixed_capacity_vector` s invalidates all iterators, and
- inserting elements at the end of an `fixed_capacity_vector` never invalidates iterators.

The following functions can potentially invalidate the iterators of `fixed_capacity_vector` s: `resize(n)` , `resize(n, v)` , `pop_back` , `erase` , and `swap` .

4.6 Naming

The following names have been considered:

- `fixed_capacity_vector` : clearly indicates that the capacity is fixed.
- `static_capacity_vector` : clearly indicates that the capacity is fixed at compile time.
- `static_vector` (Boost.Container): due to "static" / compile-time allocation of the elements. The term `static` is, however, overloaded in C++ (e.g. `static` memory?).
- `embedded_vector<T, Capacity>` : since the elements are "embedded" within the vector object itself. Sadly, the name `embedded` is overloaded, e.g., embedded systems.
- `inline_vector` : the elements are stored "inline" within the vector object itself. The term `inline` is, however, already overloaded in C++ (e.g. `inline` functions => ODR, inlining, `inline` variables).
- `stack_vector` : to denote that the elements can be stored on the stack. Is confusing since the elements can be on the stack, the heap, or the static memory segment. It also has a resemblance with `std::stack` .

4.7 Future extensions

The following extensions could be added in a backwards compatible way:

- utilities for hiding the concrete type of vector-like containers (e.g. `any_vector_ref<T>` / `any_vector<T>`).
- default-initialization of the vector elements (as opposed to value-initialization): e.g. by using a tagged constructor with a `default_initialized_t` tag.
- tagged-constructor of the form `fixed_capacity_vector(with_size_t, std::size_t N, T const& t = T())` to avoid the complexity introduced by initializer lists and braced initialization:

```
using vec_t = fixed_capacity_vector<std::size_t, Capacity>;
vec_t v0(2); // two-elements: 0, 0
vec_t v1{2}; // one-element: 2
vec_t v2(2, 1); // two-elements: 1, 1
vec_t v3{2, 1}; // two-elements: 2, 1
```

All these extensions are generally useful and not part of this proposal.

5. Technical specification

Note to editor: This enhancement is a pure header-only addition to the C++ standard library as the `<fixed_capacity_vector>` header. It belongs in the "Sequence containers" (26.3) part of the "Containers library" (26) as "Class template `fixed_capacity_vector`".

5. Class template `fixed_capacity_vector`

5.1 Class template `fixed_capacity_vector` overview

- i. A `fixed_capacity_vector` is a contiguous container that supports constant time insert and erase operations at the end; insert and erase in the middle take linear time. Its capacity is part of its type and its elements are stored within the vector object itself, meaning that that if `v` is a `fixed_capacity_vector<T, Capacity>` then it obeys the identity `&v[n] == &v[0] + n` for all `0 <= n <= v.size()`.
- ii. A `fixed_capacity_vector` satisfies all of the requirements of a container and of a reversible container (given in two tables in 26.2), of a sequence container, including the optional sequence container requirements (26.2.3), and of a contiguous container (26.2.1). The exceptions are the `push_front`, `pop_front`, and `emplace_front` member functions, which are not provided, and `swap`, which has linear complexity instead of constant complexity. Descriptions are provided here only for operations on `fixed_capacity_vector` that are not described in one of these tables or for operations where there is additional semantic information.

Note (not part of the specification): An incomplete type `T` cannot be used when instantiating the vector, as opposed to `std::vector`.

```
namespace std {

template<typename T, std::size_t C /* Capacity */>
class fixed_capacity_vector {
public:
// types:
using value_type = T;
using pointer = T*;
using const_pointer = T const*;
using reference = value_type&;
using const_reference = const value_type&;
using size_type = /*5.9 smallest unsigned integer type that can represent C (Capacity)*/;
using difference_type = std::make_signed_t<size_type>;
using iterator = implementation-defined; // see [container.requirements]
using const_iterator = implementation-defined; // see [container.requirements]
using reverse_iterator = reverse_iterator<iterator>;
using const_reverse_iterator = reverse_iterator<const_iterator>;

// 5.2, copy/move construction:
constexpr fixed_capacity_vector() noexcept;
constexpr explicit fixed_capacity_vector(size_type n);
constexpr fixed_capacity_vector(size_type n, const value_type& value);
template<class InputIterator>
constexpr fixed_capacity_vector(InputIterator first, InputIterator last);
constexpr fixed_capacity_vector(fixed_capacity_vector const& other)
    noexcept(is_nothrow_copy_constructible<value_type>{});
constexpr fixed_capacity_vector(fixed_capacity_vector&& other)
    noexcept(is_nothrow_move_constructible<value_type>{});
constexpr fixed_capacity_vector(initializer_list<value_type> il);

// 5.3, copy/move assignment:
constexpr fixed_capacity_vector& operator=(fixed_capacity_vector const& other)
    noexcept(is_nothrow_copy_assignable<value_type>{});
constexpr fixed_capacity_vector& operator=(fixed_capacity_vector&& other);
    noexcept(is_nothrow_move_assignable<value_type>{});
template<class InputIterator>
```

```

constexpr void assign(InputIterator first, InputIterator last);
constexpr void assign(size_type n, const value_type& u);
constexpr void assign(initializer_list<value_type> il);

// 5.4, destruction
/* constexpr ~fixed_capacity_vector(); */ // implicitly generated

// iterators
constexpr iterator      begin()      noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator      end()        noexcept;
constexpr const_iterator end()        const noexcept;
constexpr reverse_iterator rbegin()   noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend()     noexcept;
constexpr const_reverse_iterator rend() const noexcept;
constexpr const_iterator cbegin()     noexcept;
constexpr const_iterator cend()       const noexcept;
constexpr const_reverse_iterator crbegin() noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// 5.5, size/capacity:
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
static constexpr size_type max_size() noexcept;
static constexpr size_type capacity() noexcept;
constexpr void resize(size_type sz);
constexpr void resize(size_type sz, const value_type& c)

// 5.6, element and data access:
constexpr reference      operator[](size_type n) noexcept;
constexpr const_reference operator[](size_type n) const noexcept;
constexpr const_reference at(size_type n) const;
constexpr reference      at(size_type n);
constexpr reference      front() noexcept;
constexpr const_reference front() const noexcept;
constexpr reference      back() noexcept;
constexpr const_reference back() const noexcept;
constexpr T* data()      noexcept;
constexpr const T* data() const noexcept;

// 5.7, modifiers:
constexpr iterator insert(const_iterator position, const value_type& x);
constexpr iterator insert(const_iterator position, value_type&& x);
constexpr iterator insert(const_iterator position, size_type n, const value_type& x);
template<class InputIterator>
    constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);
constexpr iterator insert(const_iterator position, initializer_list<value_type> il);

template<class... Args>
    constexpr iterator emplace(const_iterator position, Args&&...args)
template<class... Args>
    constexpr reference emplace_back(Args&&... args);
constexpr void push_back(const value_type& x);
constexpr void push_back(value_type&& x);

constexpr void pop_back();
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);

constexpr void clear() noexcept(is_nothrow_destructible<value_type>{});

constexpr void swap(fixed_capacity_vector x)
    noexcept(noexcept(is_nothrow_swappable_v<value_type>));
};

template <typename T, std::size_t Capacity>
constexpr bool operator==(const fixed_capacity_vector<T, Capacity>& a, const fixed_capacity_vector<T, Capacity>& b) no

```

```

template <typename T, std::size_t Capacity>
constexpr bool operator!=(const fixed_capacity_vector<T, Capacity>& a, const fixed_capacity_vector<T, Capacity>& b) no
template <typename T, std::size_t Capacity>
constexpr bool operator<(const fixed_capacity_vector<T, Capacity>& a, const fixed_capacity_vector<T, Capacity>& b) no
template <typename T, std::size_t Capacity>
constexpr bool operator<=(const fixed_capacity_vector<T, Capacity>& a, const fixed_capacity_vector<T, Capacity>& b) no
template <typename T, std::size_t Capacity>
constexpr bool operator>(const fixed_capacity_vector<T, Capacity>& a, const fixed_capacity_vector<T, Capacity>& b) no
template <typename T, std::size_t Capacity>
constexpr bool operator>=(const fixed_capacity_vector<T, Capacity>& a, const fixed_capacity_vector<T, Capacity>& b) no

// 5.8, specialized algorithms:
template <typename T, std::size_t Capacity>
constexpr void swap(fixed_capacity_vector<T, Capacity>& x, fixed_capacity_vector<T, Capacity>& y)
    noexcept(noexcept(x.swap(y)));

} // namespace std

```

5.2 fixed_capacity_vector constructors

```
constexpr fixed_capacity_vector() noexcept;
```

- *Effects:* constructs an empty `fixed_capacity_vector`.
- *Complexity:* Constant.

```
constexpr explicit fixed_capacity_vector(size_type n);
```

- *Effects:* constructs a `fixed_capacity_vector` with `n` default-inserted elements.
- *Requires:* `value_type` shall be `DefaultInsertable` into `*this`.
- *Complexity:* Linear in `n`.

```
constexpr fixed_capacity_vector(size_type n, const value_type& value);
```

- *Effects:* Constructs a `fixed_capacity_vector` with `n` copies of `value`.
- *Requires:* `value_type` shall be `CopyInsertable` into `*this`.
- *Complexity:* Linear in `n`.

```
template<class InputIterator>
constexpr fixed_capacity_vector(InputIterator first, InputIterator last);
```

- *Effects:* Constructs a `fixed_capacity_vector` equal to the range `[first, last)`.
- *Requires:* `value_type` shall be `EmplaceConstructible` into `*this` from `*first`.
- *Complexity:* Initializes `distance(first, last)` `value_type`s.

5.3 Destruction

- i. The destructor shall be implicitly generated and constexpr if `is_trivial<value_type>` is true.

5.4 Size and capacity

```
static constexpr size_type capacity() noexcept;
```

- *Returns:* the total number of elements that the vector can hold.
- *Complexity:* constant.

```
static constexpr size_type max_size() noexcept;
```

- *Note:* returns `capacity()`.

```
constexpr void resize(size_type sz);
```

- *Effects:* If `sz < size()`, erases the last `size() - sz` elements from the sequence. If `sz >= size()` and `sz <= capacity()`, appends `sz - size()` copies of `c` to the sequence. The effect of calling `resize` if `sz > capacity()` is undefined.
- *Requires:* `value_type` shall be `MoveInsertable` and `DefaultInsertable` into `*this`.
- *Note:* `constexpr` if `is_trivial<value_type>`.

```
constexpr void resize(size_type sz, const value_type& c);
```

- *Effects:* If `sz < size()`, erases the last `size() - sz` elements from the sequence. If `sz >= size()` and `sz <= capacity()`, appends `sz - size()` copies of `c` to the sequence. The effect of calling `resize` if `sz > capacity()` is undefined.
- *Requires:* `value_type` shall be `MoveInsertable` and `DefaultInsertable` into `*this`.
- *Note:* `constexpr` if `is_trivial<value_type>`.

Note (not part of the specification): `resize` has as precondition: `new_size <= capacity()`. Hence it has a narrow contract, and is never `noexcept(true)`.

5.5 Element and data access

```
constexpr T* data() noexcept;  
constexpr const T* data() const noexcept;
```

- *Returns:* A pointer such that `[data(), data() + size())` is a valid range. For a non-empty vector, `data() == addressof(front())`.
- *Complexity:* Constant time.

5.6 Modifiers

```
constexpr iterator insert(const_iterator position, const value_type& x);  
constexpr iterator insert(const_iterator position, value_type&& x);  
constexpr iterator insert(const_iterator position, size_type n, const value_type& x);  
template <typename InputIterator>  
    constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);  
constexpr iterator insert(const_iterator position, initializer_list<value_type> il);
```



```
template<class... Args>
constexpr reference emplace_back(Args&&... args);
template<class... Args>
constexpr iterator emplace_back(const_iterator position, Args&&... args);
constexpr void push_back(const value_type& x);
constexpr void push_back(value_type&& x);
```

- *Remarks:* If the new size is greater than the `capacity()` the behavior is undefined. All the iterators and references before the insertion point remain valid. If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of `value_type` or by any `InputIterator` operation there are no effects. If an exception is thrown while inserting a single element at the end and `value_type` is `CopyInsertable` or `is_nothrow_move_constructible_v<value_type>` is `true`, there are no effects. Otherwise, if an exception is thrown by the move constructor of a non-`CopyInsertable` `value_type`, the effects are unspecified.
- *Complexity:* Linear in the number of elements inserted plus the distance from the insertion point to the end of the vector.
- *Throws:* Nothing unless an exception is thrown by the assignment operator or move assignment operator of `value_type`.

Note (not part of the specification): The insertion functions have as precondition `new_size <= Capacity`. Hence, they all have narrow contracts and are never `noexcept(true)`.

```
constexpr void pop_back();
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
```

- *Effects:* Invalidates iterators and references at or after the point of the erase.
- *Complexity:* The destructor of `value_type` is called the number of times equal to the number of the elements erased, but the assignment operator of `value_type` is called the number of times equal to the number of elements in the vector after the erased elements.
- *Throws:* Nothing unless an exception is thrown by the assignment operator or move assignment operator of `value_type`.

Note (not part of the specification): the erasure methods have as precondition `new_size >= 0` (always satisfied) and `new_size <= capacity`, hence they have narrow contracts.

```
constexpr void swap(fixed_capacity_vector x)
noexcept(noexcept(is_nothrow_swappable_v<value_type>));
```

- *Effects:* Exchanges the contents of `*this` with `x`. All iterators pointing to the elements of `*this` and `x` are invalidated.
- *Complexity:* Linear in the number of elements in `*this` and `x`.

5.7 fixed_capacity_vector specialized algorithms

```
template <typename T, std::size_t Capacity>
constexpr void swap(fixed_capacity_vector<T, Capacity>& x,
                    fixed_capacity_vector<T, Capacity>& y)
noexcept(noexcept(x.swap(y)));
```

- *Remarks:* This function shall not participate in overload resolution unless `Capacity == 0` or `is_swappable_v<T>` is `true`.

- *Effects*: As if by `x.swap(y)` .
- *Complexity*: Linear in the number of elements in `x` and `y` .

5.8 Zero sized `fixed_capacity_vector`

- i. `fixed_capacity_vector` shall provide support for the special case `Capacity == 0` .
- ii. In the case that `Capacity == 0` , `begin() == end() == unique value` . The return value of `data()` is unspecified.
- iii. The effect of calling `front()` or `back()` for a zero-sized array is undefined.
- iv. Non-member function `swap(fixed_capacity_vector, fixed_capacity_vector)` shall have a non-throwing exception specification.

5.9 Size type

- i. The `fixed_capacity_vector<T, Capacity>::size_type` is the smallest unsigned integer type that can represent `Capacity` .

6. Acknowledgments

The following people have significantly contributed to the development of this proposal. This proposal is based on Boost.Container's `boost::container::static_vector` and my extensive usage of this class over the years. As a consequence the authors of Boost.Container (Adam Wulkiewicz, Andrew Hundt, and Ion Gaztanaga) have had a very significant indirect impact on this proposal. The implementation of libc++ `std::vector` and the libc++ test-suite have been used extensively while prototyping this proposal, such that its author, Howard Hinnant, has had a significant indirect impact on the result of this proposal as well. The following people provided valuable feedback that influenced some aspects of this proposal: Walter Brown, Zach Laine, Rein Halbersma, and Andrzej Krzemiński. But I want to wholeheartedly acknowledge Casey Carter for taking the time to do a very detailed analysis of the whole proposal, which was invaluable and reshaped it in fundamental ways.

7. References

- [1] Boost.Container::static_vector: http://www.boost.org/doc/libs/1_59_0/doc/html/boost/container/static_vector.html .
- [2] EASTL fixed_vector: https://github.com/questor/eastl/blob/master/fixed_vector.h#L71 .
- [3] Folly small_vector: https://github.com/facebook/folly/blob/master/folly/docs/small_vector.md .
- [4] Howard Hinnant's stack_alloc: https://howardhinnant.github.io/stack_alloc.html .
- [5] P0494R0: contiguous_container proposal: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0494r0.pdf>
- [6] P0597R0: std::constexpr_vector<T> : <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0597r0.html>
- [7] P0639R0: Changing attack vector of the constexpr_vector : <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0639r0.html> .
- [8] PR0274: Clump – A Vector-like Contiguous Sequence Container with Embedded Storage: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0274r0.pdf>
- [9] Boost.Container::small_vector: http://www.boost.org/doc/libs/master/doc/html/boost/container/small_vector.html.
- [10] LLVM small_vector: http://llvm.org/docs/Doxygen/html/classllvm_1_1SmallVector.html .
- [11] EASTL design: <https://github.com/questor/eastl/blob/master/doc/EASTL%20Design.html#L284> .
- [12] Interest in StaticVector - fixed capacity vector: <https://groups.google.com/d/topic/boost-developers-archive/4n1QuJyKTTk/discussion> .
- [13] Stack-based vector container: <https://groups.google.com/d/topic/boost-developers-archive/9BEXjV8ZMeQ/discussion>.
- [14] static_vector: fixed capacity vector update: https://groups.google.com/d/topic/boost-developers-archive/d5_Kp-nmW6c/discussion.